

Билет 1. Сокеты. `socket`, `bind`, `listen`, `accept`. Синхронный однопоточный TCP-сервер

0. Формулировка билета

Сокеты. Системные вызовы `socket`, `bind`, `listen`, `accept`. Реализация синхронного однопоточного TCP-сервера.

Главная идея билета: понять, как программа-сервер открывает TCP-порт, ожидает входящие подключения, принимает клиента и обслуживает его в одном потоке.

Ключевая цепочка серверного TCP-сокета:

```
socket()
  ↓
setsockopt() // часто, но не обязательно
  ↓
bind()
  ↓
listen()
  ↓
accept()
  ↓
read() / write()
  ↓
close()
```

Для экзамена важно уметь объяснить не только код, но и смысл каждого этапа:

- `socket` создаёт сокет в ядре ОС;
- `bind` привязывает сокет к локальному адресу и порту;
- `listen` переводит сокет в пассивный режим ожидания входящих соединений;
- `accept` принимает уже установленное TCP-соединение и возвращает новый файловый дескриптор;
- `read` / `write` обмениваются байтами с клиентом;
- `close` закрывает файловый дескриптор.

1. Зачем вообще нужны сокеты

Сокет — это программная абстракция, через которую процесс может обмениваться данными с другим процессом:

- на той же машине;
- в локальной сети;
- через интернет.

В UNIX/Linux сокет представлен **файловым дескриптором**, поэтому с ним можно работать почти как с файлом:

```
read(fd, buffer, size);
write(fd, buffer, size);
close(fd);
```

Но в отличие от обычного файла, сокет связан не с данными на диске, а с сетевым соединением или сетевым адресом.

Упрощённо:

```
процесс A          процесс B
-----
socket fd  — сеть —>  socket fd
```

2. Где сокеты находятся относительно OSI/TCP-IP

В сетях часто говорят о модели OSI:

L7	Application	HTTP, FTP, SMTP
L6	Presentation	
L5	Session	
L4	Transport	TCP, UDP
L3	Network	IP
L2	Data Link	Ethernet, Wi-Fi
L1	Physical	сигналы, биты

Для прикладного программиста чаще всего важны:

```
Application layer: HTTP, свой протокол приложения
Transport layer:   TCP / UDP
Network layer:     IP
```

Сокет — это интерфейс между приложением и транспортным уровнем.

Например:

```
HTTP
↓
TCP
↓
IP
↓
Ethernet / Wi-Fi
```

Когда мы пишем TCP-сервер, мы обычно не формируем IP-пакеты руками. Мы создаём сокет, а ядро ОС само занимается TCP, IP, ретрансляциями, порядком байтов, АСК-пакетами и так далее.

3. HTTP как пример протокола поверх TCP

HTTP — это прикладной протокол. Он работает поверх TCP.

Клиент может отправить запрос:

```
GET / HTTP/1.1
Host: example.com
```

Сервер отвечает:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 13

Hello, world!
```

Важно: TCP не знает, что внутри передаются HTTP-запросы. Для TCP это просто последовательность байтов.

```
HTTP-сообщение
↓
последовательность байтов
↓
TCP-соединение
```

В учебном TCP-сервере можно не реализовывать HTTP, а просто читать байты и отправлять ответ.

4. TCP как надёжный поток байтов

TCP — это протокол транспортного уровня.

Он предоставляет приложению:

1. Двусторонний обмен

Обе стороны могут и читать, и писать.

2. Надёжность

Потерянные пакеты пересылаются заново.

3. Сохранение порядка

Если отправитель записал байты в порядке `A B C`, получатель прочитает их в том же порядке.

4. Отбрасывание дубликатов

Если пакет пришёл повторно, TCP сам это обработает.

5. Flow control

Получатель может ограничивать скорость отправителя, если не успевает читать данные.

6. Congestion control

TCP старается не перегружать сеть.

С точки зрения приложения TCP-соединение выглядит так:

```
client write() —————> server read()
client read() <———— server write()
```

Главное: **TCP — это поток байтов, а не поток сообщений.**

Если клиент сделал:

```
write(fd, "hello", 5);
```

сервер не обязан получить ровно 5 байт одним `read`.

Он может получить:

```
read #1: "he"
read #2: "llo"
```

И наоборот:

```
write(fd, "he", 2);  
write(fd, "llo", 3);
```

на другой стороне может прийти как:

```
read: "hello"
```

Отсюда важный вывод: если приложению нужны сообщения, оно само должно определить протокол.

Например:

Вариант 1. Сообщения разделяются `\n`

```
hello\nworld\n
```

Тогда читаем до символа новой строки.

Вариант 2. Сначала длина, потом тело

```
[4 байта длины][payload]
```

Например:

```
00000005 hello
```

Тогда сначала читаем длину, потом ровно столько байт тела.

5. Порт, IP, сокет, соединение

IP-адрес

IP-адрес определяет хост в сети.

Примеры:

127.0.0.1	localhost
192.168.1.10	адрес в локальной сети
0.0.0.0	специальное значение "все интерфейсы"

Порт

Порт — это 16-битное число, которое помогает ядру понять, какому процессу предназначены данные.

Пример:

```
IP: 127.0.0.1
Port: 8080
```

Вместе:

```
127.0.0.1:8080
```

Порты **0–1023** обычно считаются привилегированными и используются известными службами:

```
80    HTTP
443   HTTPS
22    SSH
```

Клиент обычно использует **эфемерный порт**, который ядро назначает автоматически.

Например:

```
client: 127.0.0.1:53124
server: 127.0.0.1:8080
```

TCP-соединение обычно идентифицируется четвёркой:

```
(src_ip, src_port, dst_ip, dst_port)
```

Например:

```
(127.0.0.1, 53124, 127.0.0.1, 8080)
```

6. Серверный и клиентский сценарий

Сервер

Сервер обычно делает:

```
socket()
bind()
listen()
accept()
read()/write()
close()
```

Клиент

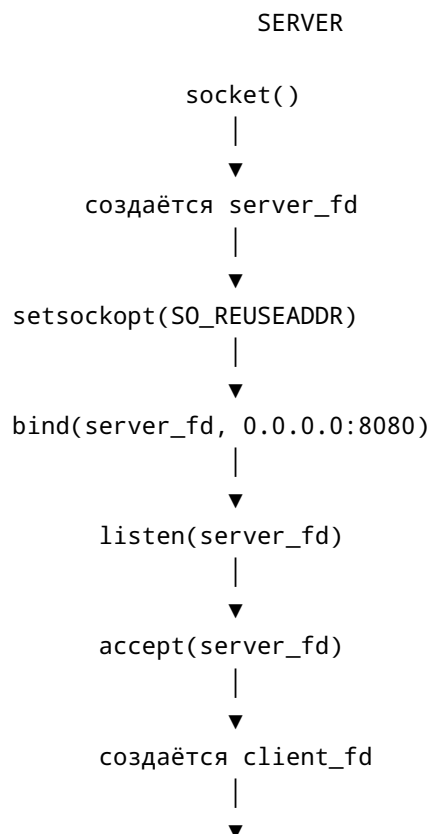
Клиент обычно делает:

```
socket()
connect()
write()/read()
close()
```

У клиента обычно нет `bind`, потому что ядро само назначает локальный эфемерный порт.

Но клиент может вызвать `bind`, если ему нужно использовать конкретный локальный порт или конкретный локальный IP.

7. Главная схема TCP-сервера



```
read(client_fd) / write(client_fd)
|
▼
close(client_fd)
```

8. Почему у сервера два разных сокета

Это один из самых важных моментов.

Есть:

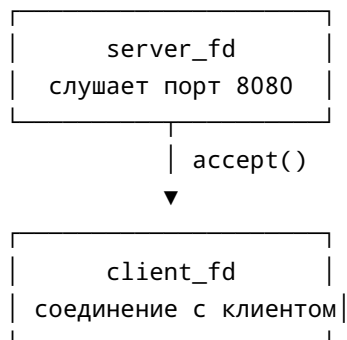
1. **Слушающий сокет** — `server_fd`.
2. **Сокет конкретного клиента** — `client_fd`.

```
server_fd
|
| accept()
▼
client_fd
```

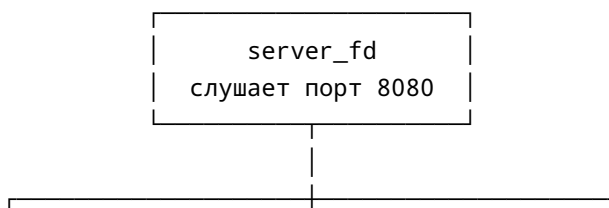
`server_fd` нужен, чтобы слушать порт и принимать новые подключения.

`client_fd` нужен, чтобы общаться с конкретным клиентом.

Схема:



Если клиентов много:



▼
client_fd #1

▼
client_fd #2

▼
client_fd #3

В однопоточном сервере мы, правда, обслуживаем их не одновременно, а последовательно.

9. socket()

Назначение

`socket()` создаёт сокет в ядре ОС и возвращает файловый дескриптор.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

Общий вид:

```
int socket(int domain, int type, int protocol);
```

Аргументы

`domain`

Семейство адресов.

Основные варианты:

```
AF_INET    // IPv4
AF_INET6   // IPv6
AF_UNIX     // локальные Unix-сокеты
```

`type`

Тип сокета.

```
SOCK_STREAM // потоковый сокет, обычно TCP
SOCK_DGRAM  // датаграммный сокет, обычно UDP
```

`protocol`

Обычно ставят `0`.

```
0
```

Это значит: выбрать протокол по умолчанию для данной пары `domain + type`.

Например:

```
AF_INET + SOCK_STREAM + 0
```

обычно означает TCP.

```
AF_INET + SOCK_DGRAM + 0
```

обычно означает UDP.

Возвращаемое значение

Если всё хорошо:

```
server_fd >= 0
```

Если ошибка:

```
server_fd == -1
```

и устанавливается `errno`.

Пример:

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

if (server_fd < 0) {
    perror("socket");
    return 1;
}
```

10. `setsockopt()` и `SO_REUSEADDR`

`setsockopt` не является центральным вызовом билета, но в реальном сервере почти всегда встречается.

```
int opt = 1;

setsockopt(
    server_fd,
```

```
SOL_SOCKET,  
SO_REUSEADDR,  
&opt,  
sizeof(opt)  
);
```

Зачем нужен `SO_REUSEADDR`

После закрытия TCP-соединения некоторые соединения могут оставаться в состоянии `TIME_WAIT`.

Если быстро перезапустить сервер, `bind` может завершиться ошибкой:

Address already in use

или:

EADDRINUSE

`SO_REUSEADDR` разрешает повторное использование локального адреса.

Обычно эту опцию ставят **до** `bind`.

```
int opt = 1;  
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));  
bind(server_fd, ...);
```

Дополнение: `SO_REUSEADDR` vs `SO_REUSEPORT`

`SO_REUSEADDR` и `SO_REUSEPORT` — разные вещи.

`SO_REUSEADDR` помогает переиспользовать адрес после перезапуска сервера.

`SO_REUSEPORT` позволяет нескольким сокетами слушать один и тот же порт, и ядро будет распределять подключения между ними.

Для базового билета достаточно знать `SO_REUSEADDR`.

11. `bind()`

Назначение

`bind` привязывает сокет к локальному адресу и порту.

```
bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
```

После `socket()` у нас есть сокет, но он ещё не связан с конкретным портом.

Чтобы сервер слушал, например, порт `8080`, нужно вызвать `bind`.

Структура `sockaddr_in`

Для IPv4 используется структура `sockaddr_in`:

```
sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;
```

Поля:

```
sin_family    // семейство адресов
sin_port      // порт
sin_addr.s_addr // IP-адрес
```

Почему нужен `htons`

Порт должен храниться в **сетевом порядке байт**.

Сетевой порядок байт — big-endian.

На большинстве обычных компьютеров порядок байт — little-endian.

Поэтому пишем:

```
addr.sin_port = htons(8080);
```

`htons` означает:

host to network short

То есть:

из порядка байт хоста в сетевой порядок для short

Обратная функция:

```
ntohs()
```

Например:

```
std::cout << ntohs(client_addr.sin_port);
```

`INADDR_ANY`

```
addr.sin_addr.s_addr = INADDR_ANY;
```

`INADDR_ANY` означает:

слушать на всех сетевых интерфейсах

То есть сервер будет принимать подключения:

```
127.0.0.1  
192.168.x.x  
другие адреса машины
```

Если нужно слушать только localhost, можно использовать:

```
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
```

Пример `bind`

```
sockaddr_in addr{};  
addr.sin_family = AF_INET;  
addr.sin_port = htons(8080);  
addr.sin_addr.s_addr = INADDR_ANY;  
  
if (bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {  
    perror("bind");  
    close(server_fd);  
    return 1;  
}
```

12. `listen()`

Назначение

`listen` переводит сокет в пассивный режим.

```
listen(server_fd, SOMAXCONN);
```

До `listen` сокет просто привязан к адресу.

После `listen` он становится **слушающим сокетом**, который может принимать входящие TCP-соединения.

`backlog`

Второй аргумент — размер очереди соединений.

```
listen(server_fd, backlog);
```

Например:

```
listen(server_fd, 128);
```

Идея:

```
клиенты подключаются
      ↓
ядро кладёт установленные соединения в очередь
      ↓
сервер достаёт их через accept()
```

Если сервер долго не вызывает `accept`, очередь может заполниться.

При переполнении новые соединения могут:

- получить отказ;
- долго ждать;
- быть сброшены;
- вести себя по-разному в зависимости от настроек ядра.

Дополнение: SYN queue и accept queue

Упрощённо в TCP-сервере можно думать о двух очередях:

SYN queue – соединения в процессе TCP handshake
accept queue – уже установленные соединения, ожидающие accept()

TCP-соединение устанавливается через three-way handshake:



`accept()` обычно возвращает соединение, для которого handshake уже завершён.

Для экзамена чаще всего достаточно сказать:

`listen` создаёт очередь входящих соединений, а `accept` достаёт из неё готовое соединение.

13. `accept()`

Назначение

`accept` принимает входящее соединение.

```
int client_fd = accept(
    server_fd,
    reinterpret_cast<sockaddr*>(&client_addr),
    &client_len
);
```

Если соединений нет, обычный blocking `accept` будет ждать.

Что возвращает `accept`

`accept` возвращает **новый файловый дескриптор**.

```
int client_fd = accept(server_fd, ...);
```

Важно:

server_fd – продолжает слушать порт
client_fd – используется для общения с конкретным клиентом

Что записывается в `client_addr`

Если передать указатель на `sockaddr_in`, ядро заполнит адрес клиента.

```
sockaddr_in client_addr{};  
socklen_t client_len = sizeof(client_addr);  
  
int client_fd = accept(  
    server_fd,  
    reinterpret_cast<sockaddr*>(&client_addr),  
    &client_len  
);
```

После этого можно вывести IP и порт клиента:

```
char client_ip[INET_ADDRSTRLEN];  
  
inet_ntop(  
    AF_INET,  
    &client_addr.sin_addr,  
    client_ip,  
    sizeof(client_ip)  
);  
  
std::cout << client_ip << ":" << ntohs(client_addr.sin_port) << "\n";
```

Почему `accept` создаёт новый сокет

Потому что слушающий сокет должен продолжать принимать новые подключения.

Если бы `server_fd` сам становился соединением с клиентом, сервер потерял бы возможность слушать порт.

Правильная модель:

```
server_fd  
├──  
├── accept() → client_fd1  
├── accept() → client_fd2  
└── accept() → client_fd3
```


14. `read()` и `write()`

После `accept` мы используем `client_fd`.

```
char buffer[1024];  
  
ssize_t n = read(client_fd, buffer, sizeof(buffer));
```

`read`

Возможные результаты:

```
n > 0
```

Прочитали `n` байт.

```
n == 0
```

Клиент закрыл соединение. Это EOF.

```
n < 0
```

Ошибка. Причину можно посмотреть через `errno`.

`write`

```
ssize_t written = write(client_fd, buffer, n);
```

Важная проблема: `write` может записать меньше байт, чем мы попросили.

Например:

```
ssize_t written = write(fd, buffer, 1000);
```

может вернуть:

```
400
```

Это значит, что записалось только 400 байт.

Поэтому профессиональнее писать вспомогательную функцию `write_all`.

```

bool write_all(int fd, const char* data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res < 0) {
            if (errno == EINTR) {
                continue;
            }

            return false;
        }

        if (res == 0) {
            return false;
        }

        written += static_cast<size_t>(res);
    }

    return true;
}

```

Дополнение: EINTR

Некоторые системные вызовы могут прерваться сигналом.

Тогда `read` или `write` может вернуть `-1`, а `errno` будет равен `EINTR`.

В таком случае часто правильно повторить вызов.

```

if (res < 0 && errno == EINTR) {
    continue;
}

```

Дополнение: SIGPIPE

Если писать в сокет, который другая сторона уже закрыла, процесс может получить сигнал `SIGPIPE`.

По умолчанию этот сигнал может завершить процесс.

В реальных серверах часто:

1. игнорируют `SIGPIPE`;
2. используют `send(..., MSG_NOSIGNAL)`;
3. корректно обрабатывают ошибку `EPIPE`.

Пример:

```
signal(SIGPIPE, SIG_IGN);
```

Но для базового учебного сервера это часто опускают.

15. `close()` и `shutdown()`

`close`

```
close(client_fd);
```

`close` закрывает файловый дескриптор.

Для TCP это обычно приводит к закрытию соединения: ядро отправляет FIN, когда это возможно.

`shutdown`

`shutdown` позволяет закрыть только одно направление соединения.

```
shutdown(client_fd, SHUT_RD); // запретить чтение
shutdown(client_fd, SHUT_WR); // запретить запись
shutdown(client_fd, SHUT_RDWR); // оба направления
```

Пример смысла `SHUT_WR` :

```
я больше не буду писать, но ещё готов читать ответ
```

Для простого сервера обычно достаточно `close`.

16. Минимальный однопоточный TCP-сервер

Это учебный вариант: он принимает клиента, читает данные, отвечает и закрывает соединение.

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <sys/socket.h>
#include <unistd.h>

int main() {
```

```

int server_fd = socket(AF_INET, SOCK_STREAM, 0);

if (server_fd < 0) {
    std::cerr << "socket failed: " << strerror(errno) << "\n";
    return 1;
}

int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) <
0) {
    std::cerr << "bind failed: " << strerror(errno) << "\n";
    close(server_fd);
    return 1;
}

if (listen(server_fd, SOMAXCONN) < 0) {
    std::cerr << "listen failed: " << strerror(errno) << "\n";
    close(server_fd);
    return 1;
}

std::cout << "Server is listening on port 8080\n";

while (true) {
    sockaddr_in client_addr{};
    socklen_t client_len = sizeof(client_addr);

    int client_fd = accept(
        server_fd,
        reinterpret_cast<sockaddr*>(&client_addr),
        &client_len
    );

    if (client_fd < 0) {
        std::cerr << "accept failed: " << strerror(errno) << "\n";
        continue;
    }

    char client_ip[INET_ADDRSTRLEN];

    inet_ntop(
        AF_INET,
        &client_addr.sin_addr,
        client_ip,

```

```

        sizeof(client_ip)
    );

    std::cout << "Accepted connection from "
                << client_ip << ":"
                << ntohs(client_addr.sin_port)
                << "\n";

    char buffer[1024];

    ssize_t n = read(client_fd, buffer, sizeof(buffer) - 1);

    if (n > 0) {
        buffer[n] = '\0';

        std::cout << "Received: " << buffer << "\n";

        const char* response = "Hello from server\n";
        write(client_fd, response, strlen(response));
    }

    close(client_fd);
}

close(server_fd);
return 0;
}

```

Что в этом коде хорошо

- есть `socket`;
- есть `SO_REUSEADDR`;
- есть `bind`;
- есть `listen`;
- есть `accept`;
- сервер различает `server_fd` и `client_fd`;
- есть базовое чтение и запись;
- клиентский сокет закрывается.

Что в этом коде упрощено

- `read` вызывается один раз;
- `write` вызывается один раз;
- не обрабатываются partial read/write;
- нет собственного протокола сообщений;
- нет обработки `EINTR`;
- нет обработки `SIGPIPE`;
- сервер однопоточный и блокирующий.

Для зачёта это допустимая база, но если хочется показать понимание, можно самому проговорить эти ограничения.

17. Улучшенный вариант: эхо-сервер с циклом чтения и write_all

Эхо-сервер отправляет клиенту обратно то, что получил.

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <sys/socket.h>
#include <unistd.h>

bool write_all(int fd, const char* data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res < 0) {
            if (errno == EINTR) {
                continue;
            }

            return false;
        }

        if (res == 0) {
            return false;
        }

        written += static_cast<size_t>(res);
    }

    return true;
}

void handle_client(int client_fd) {
    char buffer[4096];

    while (true) {
        ssize_t n = read(client_fd, buffer, sizeof(buffer));

        if (n < 0) {
            if (errno == EINTR) {
                continue;
            }

            std::cerr << "read failed: " << strerror(errno) << "\n";
        }
    }
}
```

```

        break;
    }

    if (n == 0) {
        std::cout << "client closed connection\n";
        break;
    }

    if (!write_all(client_fd, buffer, static_cast<size_t>(n))) {
        std::cerr << "write failed: " << strerror(errno) << "\n";
        break;
    }
}

close(client_fd);
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        std::cerr << "socket failed: " << strerror(errno) << "\n";
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) <
0) {
        std::cerr << "bind failed: " << strerror(errno) << "\n";
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, SOMAXCONN) < 0) {
        std::cerr << "listen failed: " << strerror(errno) << "\n";
        close(server_fd);
        return 1;
    }

    while (true) {
        sockaddr_in client_addr{};
        socklen_t client_len = sizeof(client_addr);

        int client_fd = accept(

```

```

        server_fd,
        reinterpret_cast<sockaddr*>(&client_addr),
        &client_len
    );

    if (client_fd < 0) {
        if (errno == EINTR) {
            continue;
        }

        std::cerr << "accept failed: " << strerror(errno) << "\n";
        continue;
    }

    handle_client(client_fd);
}

close(server_fd);
}

```

Этот сервер всё ещё однопоточный.

Главный недостаток:

пока `handle_client(client_fd)` обслуживает одного клиента, сервер не вызывает `accept()` для новых клиентов.

То есть один медленный клиент блокирует обслуживание остальных.

18. Почему однопоточный сервер плохо масштабируется

Однопоточный синхронный сервер делает всё последовательно.

```

accept client #1
handle client #1
close client #1

accept client #2
handle client #2
close client #2

accept client #3
handle client #3
close client #3

```

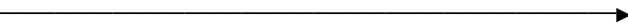
Если клиент #1 подключился и ничего не отправляет, сервер может зависнуть на:


```
read(client_fd, buffer, sizeof(buffer));
```

В это время клиент #2 не будет обработан.

Даже если клиент #2 уже установил TCP-соединение, он будет ждать, пока сервер вернётся к `accept`.

Схема:

time 

server: accept #1 — read #1 — read #1 — read #1 — ...
client #1: подключился и молчит
client #2: подключился, но сервер его не обслуживает
client #3: подключился, но сервер его не обслуживает

Отсюда естественный переход к следующим билетам:

```
однопоточный сервер
    ↓
многопоточный сервер
    ↓
thread pool
    ↓
non-blocking I/O
    ↓
select / poll / epoll
    ↓
io_uring
```

19. Тестирование сервера

nc

Самый простой способ:

```
nc localhost 8080
```

Пишем текст, сервер отвечает.

`telnet`

```
telnet localhost 8080
```

`telnet` старый, но иногда удобен для ручной проверки TCP.

`curl`

Если сервер отвечает в HTTP-формате, можно использовать:

```
curl -v http://localhost:8080
```

Но простой echo-сервер не обязан быть HTTP-сервером.

`ss`

Посмотреть слушающие TCP-сокеты:

```
ss -tln
```

Посмотреть TCP-соединения:

```
ss -tn
```

`netstat`

Старый аналог:

```
netstat -tln
```

20. TCP-состояния, которые полезно знать

Для базового билета не обязательно глубоко знать автомат TCP, но полезно понимать основные состояния.

`LISTEN`

Серверный сокет после `listen`.

```
server_fd находится в LISTEN
```

SYN_SENT

Клиент отправил SYN и ждёт ответа.

SYN_RCVD

Сервер получил SYN, отправил SYN+ACK и ждёт ACK.

ESTABLISHED

Соединение установлено.

`accept` возвращает соединение, которое уже готово к обмену данными.

TIME_WAIT

Состояние после закрытия соединения. Нужно, чтобы корректно обработать запоздавшие пакеты и финальное закрытие TCP-соединения.

Именно из-за `TIME_WAIT` при быстром перезапуске сервера иногда возникает `EADDRINUSE`, поэтому полезен `SO_REUSEADDR`.

21. UDP и Unix-сокеты как контекст

Хотя билет 1 про TCP, в конспекте полезно сравнить TCP с другими сокетами.

TCP

```
socket(AF_INET, SOCK_STREAM, 0);
```

Свойства:

- есть соединение;
- есть `listen`;
- есть `accept`;
- есть `connect`;
- надёжная доставка;
- сохранение порядка;
- поток байтов.

UDP

```
socket(AF_INET, SOCK_DGRAM, 0);
```

Свойства:

- нет соединения в TCP-смысле;

- обычно нет `listen` и `accept`;
- используются `sendto` и `recvfrom`;
- нет гарантии доставки;
- нет гарантии порядка;
- сохраняются границы датаграмм.

Аналогия:

TCP – телефонный разговор
UDP – открытки

Unix-сокеты

```
socket(AF_UNIX, SOCK_STREAM, 0);
```

Используются для взаимодействия процессов на одной машине.

Плюсы:

- быстрее сетевых сокетов;
- не нужен IP;
- можно использовать путь в файловой системе;
- можно передавать файловые дескрипторы.

22. Частые ошибки в ответе

Ошибка 1. “`accept` создаёт новый серверный сокет”

Лучше говорить:

`accept` создаёт новый сокет для общения с конкретным клиентом. Слушающий сокет остаётся слушать порт.

Ошибка 2. “TCP передаёт сообщения”

TCP передаёт поток байтов, а не сообщения.

Если нужны сообщения, нужен протокол приложения:

- разделитель;
 - длина перед сообщением;
 - фиксированный размер;
 - HTTP-like формат.
-

Ошибка 3. Забыть `htons`

Неправильно:

```
addr.sin_port = 8080;
```

Правильно:

```
addr.sin_port = htons(8080);
```

Ошибка 4. Не понимать `INADDR_ANY`

`INADDR_ANY` — это не адрес клиента.

Это специальное значение для сервера:

принимать соединения на всех интерфейсах

Ошибка 5. Не проверять ошибки системных вызовов

Почти все системные вызовы могут вернуть ошибку:

```
socket  -> -1
bind    -> -1
listen  -> -1
accept  -> -1
read    -> -1
write   -> -1
```

На экзамене не обязательно писать промышленный error handling, но нужно понимать, что ошибки бывают.

Ошибка 6. Не понимать, почему сервер однопоточный

Синхронный однопоточный сервер:

```
один поток
blocking accept
blocking read
blocking write
```

Он не может одновременно обслуживать нескольких клиентов.

Ошибка 7. Использовать `printf("%s", buffer)` после `read` без `\0`

`read` читает байты, а не C-строку.

Если хотим печатать как строку, нужно добавить нуль-терминатор:

```
ssize_t n = read(fd, buffer, sizeof(buffer) - 1);

if (n > 0) {
    buffer[n] = '\0';
    printf("%s\n", buffer);
}
```

Но это работает только если данные действительно текстовые.

23. Что сказать на экзамене: короткая версия

Можно ответить так:

Сокет — это абстракция одного конца сетевого соединения. В UNIX он представлен файловым дескриптором, поэтому с ним можно работать через `read`, `write`, `close`.

Для TCP-сервера сначала вызываем `socket(AF_INET, SOCK_STREAM, 0)`. `AF_INET` означает IPv4, `SOCK_STREAM` — потоковый сокет, то есть TCP. Затем обычно ставим `SO_REUSEADDR`, чтобы можно было быстро перезапустить сервер и не получить `EADDRINUSE`.

Потом заполняем `sockaddr_in`: указываем `sin_family = AF_INET`, порт через `htons(port)` и адрес, например `INADDR_ANY`, если хотим слушать на всех интерфейсах. Через `bind` привязываем сокет к адресу и порту.

После этого вызываем `listen`. Сокет становится пассивным, то есть слушающим. Вторым аргумент `listen` — это `backlog`, размер очереди установленных соединений, ожидающих `accept`.

Затем в цикле вызываем `accept`. Если клиентов нет, blocking `accept` ждёт. Когда соединение появилось, `accept` возвращает новый файловый дескриптор `client_fd`. Важно, что `server_fd` продолжает слушать порт, а `client_fd` используется для общения с конкретным клиентом.

Дальше сервер читает данные через `read(client_fd, ...)`, пишет ответ через `write(client_fd, ...)` и закрывает `client_fd`. TCP — это поток байтов, поэтому `read` и `write` могут обработать меньше байт, чем мы ожидали. В

профессиональном коде нужно делать циклы чтения/записи и иметь протокол сообщений.

Такой сервер называется синхронным и однопоточным, потому что все операции блокирующие и выполняются в одном потоке. Пока сервер обслуживает одного клиента, остальные ждут. Это главный недостаток такого подхода и мотивация для многопоточного сервера, thread pool и неблокирующего I/O.

24. Мини-код для ответа на доске

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;

bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
listen(server_fd, SOMAXCONN);

while (true) {
    sockaddr_in client_addr{};
    socklen_t client_len = sizeof(client_addr);

    int client_fd = accept(
        server_fd,
        reinterpret_cast<sockaddr*>(&client_addr),
        &client_len
    );

    char buffer[1024];

    ssize_t n = read(client_fd, buffer, sizeof(buffer));

    if (n > 0) {
        write(client_fd, buffer, n);
    }

    close(client_fd);
}
```

Что обязательно объяснить к этому коду:

```
server_fd — слушающий сокет
client_fd — сокет конкретного клиента
```

htons — перевод порта в сетевой порядок байт
INADDR_ANY — слушаем все интерфейсы
accept — блокируется и возвращает новый fd
read/write — работают с потоком байтов
close — закрывает соединение с клиентом

25. Мини-проверка

1. Почему серверу нужны два файловых дескриптора: `server_fd` и `client_fd`?
2. Что делает `bind`?
3. Что делает `listen`?
4. Что такое `backlog`?
5. Что делает `accept`, если клиентов нет?
6. Почему `accept` возвращает новый сокет?
7. Почему порт нужно оборачивать в `htons`?
8. Что означает `INADDR_ANY`?
9. Почему TCP называют потоком байтов?
10. Может ли `read` прочитать меньше байт, чем отправил клиент?
11. Может ли `write` записать меньше байт, чем мы попросили?
12. Что означает `read == 0`?
13. Зачем нужен `SO_REUSEADDR`?
14. Чем `close` отличается от `shutdown`?
15. Почему однопоточный сервер плохо масштабируется?
16. Чем TCP отличается от UDP с точки зрения сокетов?
17. Почему простой echo-сервер не является HTTP-сервером?
18. Что произойдёт, если клиент подключился и ничего не отправляет?
19. Что такое эфемерный порт клиента?
20. Как протестировать сервер через `nc`?

26. Самый короткий ответ, если времени совсем мало

TCP-сервер создаётся цепочкой `socket → bind → listen → accept`.

`socket(AF_INET, SOCK_STREAM, 0)` создаёт TCP-сокет.

`bind` привязывает его к IP и порту, порт указывается в сетевом порядке байт через `htons`.

`listen` переводит сокет в режим ожидания входящих соединений.

`accept` блокируется до появления клиента и возвращает новый fd для конкретного соединения.

Слушающий `server_fd` остаётся слушать порт, а `client_fd` используется для `read/write`.

TCP — поток байтов, поэтому границы сообщений не сохраняются, а `read/write` могут обработать меньше байт, чем ожидалось.

Однопоточный сервер обслуживает клиентов последовательно, поэтому один медленный клиент может заблокировать всех остальных.